Design a Search Autocomplete System

Typeahead / search-as-you-type — the "Top K" problem

Key takeaways — System Design Interview, Chapter 13

Step 1 — Clarify the Requirements

Fast	Return results within ~100ms, or it feels laggy
Relevant	Suggestions match the prefix (match at the start only)
Sorted	Ranked by historical query frequency; return top 5
Scalable	Handle high traffic (10M DAU)
Highly available	Stay usable even when parts of the system fail

Back-of-the-Envelope Estimation

10M

Daily active users
10 searches per user/day

~24,000

Average QPS
Peak ×2 ≈ 48,000

20 bytes

Per query
1 request per character typed

0.4 GB

New data per day
(assuming 20% queries are new)

Step 2 — High-Level Design: Two Services

Data Gathering

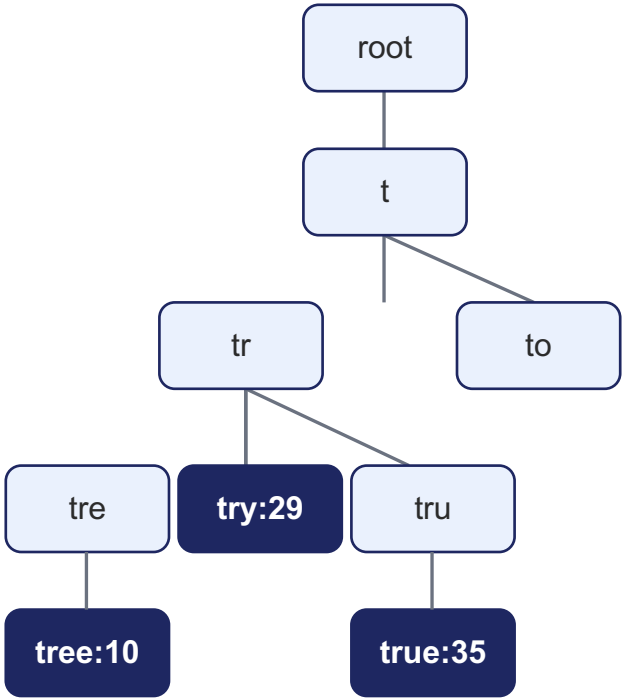
- Collects user input, aggregates frequency
- Maintains a "query → frequency" table
- Not real-time; processed in periodic batches

Query Service

- Given a prefix, return top 5 queries
- Naive v1: SQL LIKE 'prefix%', sort, take 5
- DB becomes a bottleneck at scale → use a Trie

Core Data Structure: the Trie (Prefix Tree)

- Compactly stores strings; built for prefix lookup
- Root = empty string; each node = a prefix
- Nodes store frequency to rank popularity
- Query: find prefix → walk subtree → take top k
- **Downside: worst case walks the whole subtree — slow**



Type "tr" → top 2 = [true: 35, try: 29]

Trie Optimizations: Get Queries to O(1)

1. Limit prefix length

Users rarely type long queries. Cap p at a small constant (e.g. 50), so "find prefix" drops from O(p) to O(1).

2. Cache Top K per node

Store the top 5 queries on each node, so there's no subtree walk. Trade space for time — response speed matters most.

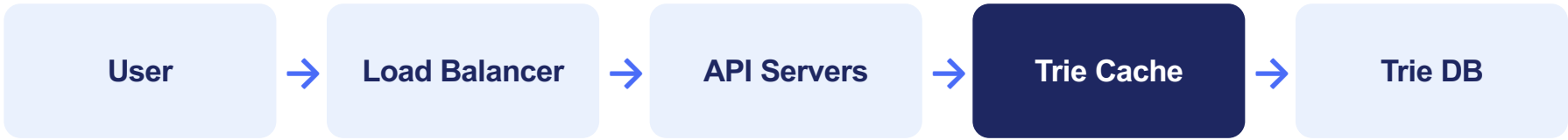
Result: Find prefix O(1) + read cached Top K O(1) → overall query is **O(1)**

Data Gathering: Offline Batch Pipeline

Don't rebuild the Trie on every query (too slow, unnecessary). Rebuild periodically — e.g. weekly.

- Key-value mapping: prefix = key, node's Top K = value (e.g. be → [best:35, bet:29, bee:20…])

Query Service: Read Path & Speedups



On cache miss, fall back to Trie DB and backfill the cache

AJAX requests — Send/receive without reloading the whole page

Browser caching — Suggestions change little; cache client-side (Google caches 1 hour)

Data sampling — Don't log every query — e.g. log 1 in N to save compute & storage

Trie Operations & Content Filtering

Create

Workers build the Trie from aggregated data; source is the analytics logs

Update

Option 1: rebuild the whole Trie weekly, replace the old one (preferred)
Option 2: update a single node + all its ancestors (slow; only for small Tries)

Delete

Hateful, violent, or explicit suggestions must be removed:
add a filter layer in front of Trie Cache to block them, then physically delete from the DB before the next rebuild

Scaling Storage: Sharding

Simple approach: shard by first letter

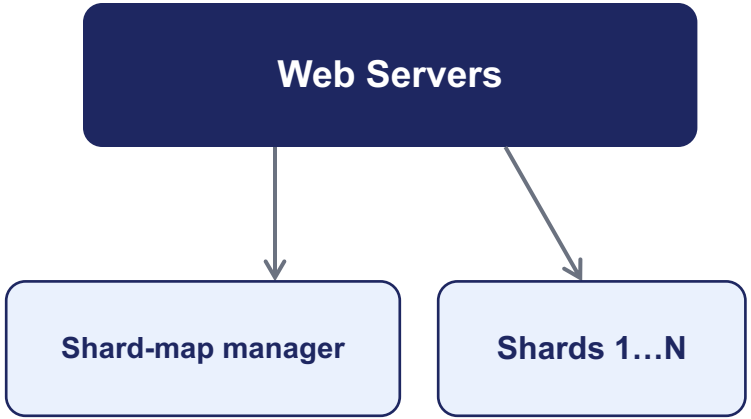
2 servers: a–m / n–z; up to 26; sub-shard further (aa–ag…)

Problem: uneven distribution

Far more words start with 'c' than 'x' — load skews badly

Fix: smart sharding

Analyze history; a shard-map manager decides where each row lives (e.g. 's' alone, 'u–z' together)



Step 4 — Wrap-Up & Follow-Ups

Multi-language support?	Store Unicode characters in Trie nodes to cover all writing systems
Queries differ by country?	Build a separate Trie per country; store in a CDN to cut latency
Trending (real-time) queries?	Weekly rebuilds can't keep up with viral spikes: shrink the working set via sharding, reweight ranking toward recent queries, use stream processing (Kafka / Spark Streaming)

Remember: Trie + per-node Top K cache = O(1) queries; offline batch rebuild + cache = scalability